

Programación de Servicios y Procesos
UT04- Generación de servicios en red
Tarea 5.1

Alumna:
Valentina Alessandra Calicchia.

Marzo, 2025.

Implementación de los Ejercicios 1 y 2 en el Servidor HTTP

Este documento explica las modificaciones realizadas en el proyecto Java ServerHTTP para cumplir con los siguientes requisitos:

- Ejercicio 1: Incluir la cabecera Date en las respuestas HTTP.
- Ejercicio 2: Implementar multihilo para gestionar la concurrencia de manera eficiente.

Ejercicio 1: Inclusión de la cabecera Date

Para cumplir con este requerimiento, se ha agregado la cabecera Date en la respuesta HTTP. Esta cabecera indica la fecha y hora del servidor en el momento de la respuesta.

Modificaciones realizadas

En la clase ServidorHTTP, dentro del método procesaPetición, se ha agregado la siguiente línea antes de enviar el contenido de la respuesta:

```
import java.time.LocalDateTime;

// Añadimos la cabecera Date:
printWriter.println("Date: " + LocalDateTime.now());
```

Esta línea se ha agregado en los tres casos de respuesta:

- Cuando se solicita la página de inicio (/).
- Cuando se solicita la página del Quijote (/quijote).
- Cuando la página solicitada no existe (error 404).

Esta modificación permite que cada respuesta HTTP devuelta por el servidor incluya la fecha y hora exacta en la que fue generada.

Ejercicio 2: Implementación de multihilo

Para mejorar el rendimiento y permitir que el servidor atienda múltiples clientes simultáneamente, se ha implementado el manejo de peticiones utilizando hilos.

Modificaciones realizadas

1. Creación de una clase Hilo

Se ha creado una nueva clase Hilo que implementa la interfaz Runnable. Esta clase se encarga de gestionar cada petición de manera independiente:

```
package PaquetePrincipal;
```

```

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;
import java.util.logging.Level;
import java.util.logging.Logger;

/**
 *
 * @author valen
 */
public class Hilo implements Runnable {

    private final Socket socCliente;

    public Hilo(Socket socCliente) {
        this.socCliente = socCliente;
    }

    @Override
    public void run() {
        System.out.println("Atendiendo al cliente...");

        try (
            InputStreamReader inSR = new
InputStreamReader(socCliente.getInputStream());
            BufferedReader bufLeer = new BufferedReader(inSR);
            PrintWriter printWriter = new
PrintWriter(socCliente.getOutputStream(), true)
        ) {
            String peticion = bufLeer.readLine();

            if (peticion == null || !peticion.startsWith("GET")) {
                return;
            }

            // Extraer ruta de la petición GET
            String ruta = peticion.split(" ")[1]; // Extrae lo que está entre
"GET" y "HTTP/1.1"

            switch (ruta) {
                case "/":
                    responderCliente(printWriter, Paginas.html_index,
Mensajes.lineaInicial_OK);

```

```

        break;
        case "/quijote":
            responderCliente(printWriter,      Paginas.html_quijote,
Mensajes.lineaInicial_OK);
            break;
        default:
            responderCliente(printWriter,      Paginas.html_noEncontrado,
Mensajes.lineaInicial_NotFound);
            break;
    }

    } catch (IOException ex) {
        Logger.getLogger(Hilo.class.getName()).log(Level.SEVERE, "Error en
la conexión con el cliente", ex);
    } finally {
        try {
            socCliente.close();
            System.out.println("Cliente atendido y conexión cerrada.");
        } catch (IOException ex) {
            Logger.getLogger(Hilo.class.getName()).log(Level.SEVERE,
"Error cerrando el socket", ex);
        }
    }
}

private void responderCliente(PrintWriter printWriter, String html, String
status) {
    printWriter.println(status);
    printWriter.println(Paginas.primerCabecera);
    printWriter.println("Content-Length: " + html.length());
    printWriter.println(); // Línea en blanco obligatoria en HTTP
    printWriter.println(html);
}
}

```

1. Modificación de la clase ServidorHTTP

En lugar de procesar las peticiones en el mismo hilo principal, ahora cada conexión entrante es manejada por un nuevo hilo:

```
while (true) {
```

```
// A la espera de peticiones
socCliente = socServidor.accept();
System.out.println("Atendiendo a un nuevo cliente...");

// Se crea un nuevo hilo para procesar la petición
Thread hiloCliente = new Thread(new Hilo(socCliente));
hiloCliente.start();
}
```

Beneficios de la implementación multihilo

- Permite que múltiples clientes sean atendidos simultáneamente.
- Evita bloqueos en la ejecución del servidor cuando hay peticiones concurrentes.
- Mejora la escalabilidad del servidor.

Conclusiones

Con estas modificaciones, el servidor HTTP ahora:

1. Incluye la cabecera Date en las respuestas HTTP, proporcionando información de tiempo en cada petición.
2. Soporta concurrencia utilizando hilos, permitiendo atender múltiples clientes al mismo tiempo de manera eficiente.

Esta implementación optimiza el rendimiento del servidor y lo hace más robusto para manejar conexiones concurrentes.